

HW4 — Static + Dynamic Scraping (BeautifulSoup, Playwright)

Overview

If you try to scrape on Anthropic's careers page and you'll get back an HTML shell with no job listings in it at all. The list is built entirely in the browser by JavaScript after the page loads via user interactions. This homework adds **Playwright** to handle that case, and merges its results with the Vertex AI Search path from HW2 into one combined dataset.

In addition, each job post including Google, Anthropic and OpenAI is displayed in a static website. Those pages include candidates' qualifications, which could be parsed by **BeautifulSoup**.

Learning Objectives

- Automate a real browser with Playwright: navigate, wait for elements, click elements, and read the resulting DOM — using the sync API, or the async API if you want the optional speedup.
- Scrape a static HTML page with `requests` + `BeautifulSoup`, handling real-world markup inconsistencies (headings that aren't heading tags, empty lists, nav menus that look like content).
- Recognize when a site needs a real browser vs. a plain HTTP request — server-rendered vs. client-rendered pages.
- Merge two independent, differently-shaped data sources into one consistent output without silently dropping either one.
- Extend a Docker image to include a headless browser and its dependencies.

Background: why Anthropic needs a different approach

`requests.get()` only ever sees the raw HTML, and it never runs JavaScript to filter output by the page's search box entry. Playwright solves this by actually launching a browser, loading the page, waiting for JavaScript to finish rendering, and then reading the DOM.

Filling the search box does not navigate anywhere — React re-renders the list in place, a few milliseconds later. `wait_for_selector` on a job-listing element returns as soon as *one* exists, and one already existed. Wait for a listing to exist and then pause briefly before reading anything. `test_hw4_api.py` checks the returned titles against the role you searched for, which is exactly how this bug shows up.

Background: career-page markup is not uniform

On the static pages you'll parse with BeautifulSoup, qualifications are usually a heading followed by a bulleted list:

```
<h3><strong>In this role, you will:</strong></h3>
<ul>
  <li>Design, build, and maintain data pipelines...</li>
  <li>Partner with cross-functional teams...</li>
</ul>
```

But not always. Some pages use a `<p>` with bold text instead of a real heading tag, and plenty of `<ul>`s on a page have nothing to do with qualifications at all — nav menus, footers, office lists. That is why `retreive_qualification()` below is selective rather than just "grab every `<ul>`".

Provided Files

- `fastapi/gemini_summarizer.py` — complete. It uses the Gemini [Batch API](#): `return_gemini_summaries()` submits every qualification as one job instead of one interactive call per posting to overcome the rate limits.
- `fastapi/extract_save_data.py` — `add_qualification()`, `parse_google_search_results()`, `call_google_search()` and `save_to_gcs()` are all given to you complete. Only `search_and_save_jobs()` is a stub (see below).
- `fastapi/user_definition.py` and `fastapi/requirements.txt` (which now has `playwright`, `beautifulsoup4` and `requests`).
- `streamlit/hw4.py`, `streamlit/user_definition.py`, and `streamlit/requirements.txt`.

Bring these over from your HW3

HW4 deploys exactly the way HW3 did — two Cloud Run services plus a Cloud Scheduler job — so the three files HW3 asked you for come forward into HW4. What ships in this folder are one-line `# TODO: Add/Update your HW3` placeholders that only mark where each file goes:

File	What to do
<code>fastapi/Dockerfile</code>	Copy your HW3 version, then add the Playwright line below
<code>streamlit/Dockerfile</code>	Copy your HW3 version as is
<code>gcloud_command.sh</code>	Copy your HW3 version, then apply the changes below

`config.sh` is **not** shipped with HW4 either — copy your HW3 `config.sh` into this folder so `gcloud_command.sh` can `source` it. It needs no edits.

Files You Must Complete

`fastapi/qualification_parser.py` (complete the provided stub)

This is the file HW3 no longer asks for, so it is new work here. The stub gives you `QUALIFICATION_KEYWORDS` and the browser-like `User-Agent` header (some career pages answer a header-less request with a `403`); the parsing is yours.

`retreive_qualification(url: str) -> dict` — fetch the page, find every `<ul>`, and for each one:

1. Find its nearest preceding heading ("h2", "h3", "h4", "p"). If there is none, skip the `<ul>`.
2. If that heading is a `<p>`, only accept it when it has a `<strong>/<b>` child, and use that child's text as the heading text. A bare `<p>` above a list is body copy, not a heading. For a real heading tag, use the heading's own text.
3. Skip the `<ul>` if it has no `<li>` children.

4. Skip it if the **lower-cased** heading text contains none of `QUALIFICATION_KEYWORDS`. This is what keeps nav menus, footers and office-location lists out of your results.
5. Otherwise collect the text of every `<li>` and store the list under the heading text as the dict key.

Return a dictionary shaped like:

```
{header_string1: [list_of_qualification],
 header_string2: [list_of_qualification], ...}
```

Two things to get right:

- **Return {} when nothing matches, never None.**
- **This function is now called from two places.** `add_qualification()` uses it for the Vertex AI Search results, and your `retrieve_anthropic_jobs()` calls it per Anthropic posting. Anthropic's postings live on Greenhouse, whose markup differs from Google's and OpenAI's.

`fastapi/anthropic_search.py` (complete the provided stub)

def retrieve_anthropic_jobs(url: str, role: str) -> list Write this as an ordinary function using Playwright's **sync** API. An **async** `def` version driving `playwright.async_api` is **optional** — it's faster when a search turns up many postings, but it is not required, and `test_hw4_api.py` accepts either one. Two things to watch if you take the sync route: the stub's import line is `from playwright.async_api import async_playwright`, so change it to `from playwright.sync_api import sync_playwright`, and drop the `asyncio.run(...)` wrapper when you call it from `extract_save_data.py`.

The steps below are written with **await**s, for the async version; for the sync version, do the same things without them.

1. Normalize `url` — `company_dictionary` values elsewhere in this project are stored without a scheme (e.g. `"openai.com/careers"`), but `page.goto()` needs a full URL. Prepend `https://` if it's missing.
2. `with sync_playwright() as p:` (or `async with async_playwright()`) — launch Chromium (`headless=True` for containers), open a new page, `goto(url)`.
3. `page.wait_for_selector("main")`, then find the search input with `page.get_by_placeholder("Search roles")` and `.fill(role)`.
4. Wait for at least one job-listing element to exist, then an additional short pause (see the race-condition note above) before reading anything.
5. Find every job-listing link — inspect the page's actual HTML to find a CSS selector. Career pages built with CSS modules often have hashed class names like `JobsPage-module-scss-module__8DFgsW__jobItem` — match the stable substring with an attribute-contains selector, `a[class*="jobItem"]`, rather than the exact hashed class.
6. For each job link, pull its `href` and title text and call `retrieve_qualification()` on the link.
7. After the loop, pass every qualification you collected to `return_gemini_summaries()` in one call. It submits them as a single Gemini batch job and returns one skills list per qualification, in the same order, so `zip()` the two lists back together.

- 8. Return a list of {"title", "link", "date", "qualification", "skills"}dicts. `close()` the browser before returning.

`fastapi/extract_save_data.py` (complete the provided stub)

`search_and_save_jobs` is the only stub left in this file — everything else is provided. It needs to route "Anthropic" to `retrieve_anthropic_jobs()` instead of the Vertex AI Search path, send every other company through `call_google_search()`, and merge both result sets into one response before saving to GCS. The numbered steps are in the stub's docstring.

`fastapi/Dockerfile` and `streamlit/Dockerfile` (use or extend your HW3 version)

For fastapi, add one line after `pip install -r requirements.txt`:

```
RUN playwright install --with-deps chromium
```

The `playwright pip` package only installs the Python driver — the actual Chromium **browser binary** have to be installed separately. `--with-deps` installs system packages. Skipping this step doesn't cause a build failure — it fails later, at runtime, when `p.chromium.launch()` can't find or run the browser.

The image also gets noticeably bigger — Chromium plus its system libraries is a few hundred MB — so expect a slower first Cloud Build.

`gcloud_command.sh` (extend your HW3 version)

Start from the `gcloud_command.sh` you submitted for HW3, with every `FILL_IN_<n>` already resolved, and drop it in this folder. The five steps and all of the HW3 blanks stay exactly as they were. What changes is that **the FastAPI service now runs a browser**, and a service sized for "receive JSON, call an API, return JSON" will not survive that. All three of these are on **STEP 2's gcloud run deploy for the API service only** — the Streamlit service never launches a browser and needs none of them.

Add to STEP 2	Why
<code>--memory</code> well above the default	Cloud Run defaults to 512Mi . Chromium alone wants more than that, and a page with a full job list wants more again. 2Gi is a safe starting point.
<code>--cpu</code> of at least 2	A browser rendering a React app on a single shared vCPU is what turns <code>wait_for_selector</code> into a <code>TimeoutError</code> .
<code>--timeout</code> raised	The default request timeout is 300s (5 min) . One request now launches a browser, scrapes every matching posting with <code>requests</code> , and waits on a Gemini batch job — <code>gemini_summarizer.py</code> polls for up to 15 minutes on its own (<code>MAX_WAIT_SECONDS</code>). Cloud Run allows up to 3600 .

And one change to **STEP 5's scheduler job**:

Add to STEP 5	Why
-- attempt-- deadline	Cloud Scheduler gives up on an HTTP target after 180s by default and then <i>retries</i> — which starts a second browser scrape while the first is still running. The maximum is 1800s .

How these fail is the point. None of them break `docker build`, and none of them break a local `docker run` where your laptop has plenty of RAM. They break in Cloud Run, at request time:

- Too little memory : the container is OOM-killed mid-request. You get a **500** or a dropped connection, and the words "out of memory" appear only in the Cloud Run logs, not in the response.
- Too short a timeout : a **504** after exactly 5 minutes, with the search having actually worked right up until Cloud Run cut it off.
- Default attempt deadline : the scheduler job shows as failed every night even though the GCS blob gets written, because the write finished after Scheduler stopped listening.

So check the deployed API by POSTing to it and reading the Cloud Run logs, not by watching `gcloud_command.sh` exit 0.

How to Run

Work locally first — a browser bug is much cheaper to find on your laptop than in Cloud Run.

```
cd fastapi
pytest test_hw4_api.py -k retrieve_qualification # your parser first
pytest test_hw4_api.py                        # everything, incl.
Playwright (slow, live)
```

Then build and run the containers, and only once a local POST works, deploy:

```
bash gcloud_command.sh
```

Note that a single search now takes minutes, not seconds: one browser run, one `requests.get()` per posting, and a Gemini batch job that is queued rather than answered immediately.

Provided Tests

`fastapi/test_hw4_api.py` — it covers both `qualification_parser.py` and `anthropic_search.py`. It lives in `fastapi/`, so run `pytest` from there. `pycodestyle` has to be installed for the PEP8 checks at the bottom of the file.

`qualification_parser.py` tests (run these first)

- **Live tests** against a real, currently-open OpenAI posting and Google posting. Like any test that depends on an external site, these go stale when a posting closes or a page is redesigned — if one starts failing, open the URL in the test file and check it still goes to a real posting with a qualifications section before assuming your code is broken.
- **Offline parsing tests** that swap out `requests.get` and run your parser against fixed HTML, so they need no network and a failure is always your code. They pin down each rule separately: a real heading plus its bullets is collected; a `<p><strong>` heading contributes the **bold child's** text as the key; a bare `<p>` above a list is skipped; a `<ul>` with no `<li>` is skipped; a heading with no keyword is skipped; keyword matching is case-insensitive; several qualification sections on one page are all collected; and a page with no qualifications returns `{}` rather than `None`. They also assert your request still sends a `User-Agent` header.

A parser that fails these will also fail the Anthropic qualification check below, but with a much slower speed and lengthy error — so filter them with `-k retrieve_qualification` before running the whole file.

anthropic_search.py tests

Live tests that launch a real headless Chromium and scrape Anthropic's actual careers page.

- **Either signature is accepted.** A helper calls `retrieve_anthropic_jobs()` and awaits the result only if it got a coroutine back, so the required sync version and the optional async version both pass.
- **The postings are real.** Every result has `title`, `link`, `date`, `qualification` and `skills`; the link is an absolute `https://...greenhouse.io/anthropic/...` URL (relative hrefs break `retrieve_qualification()`, which feeds the link straight to `requests.get()`); `date` is today.
- **The search box actually filtered.** If you read the DOM before React finishes re-rendering, you get the full unfiltered job list - the titles are checked against the role searched for.
- **Qualifications were parsed.** Each `qualification` must be a `{heading: [bullet, ...]}` dict, and at least one posting must have a non-empty one. All-empty dicts mean the per-posting scrape silently failed.
- **Skills were summarized.** A `None` for one job is tolerated, but not for all of them.
- **The summarizer is called once, not once per posting.** This test monkeypatches both `retrieve_qualification` and `return_gemini_summaries` on the `anthropic_search` module and asserts the summarizer got exactly one call holding every qualification.
- **URL-scheme handling**, via a role with zero matches, so it verifies normalization in seconds rather than re-running the expensive path.
- **return_gemini_summaries() directly** -- one summary per input, in the same input order (`retrieve_anthropic_jobs()` zips them back onto the jobs by position), and `[]` for an empty input.

Submission Checklist

- ☐ `retrieve_qualification` skips `<ul>`s with no `<li>`s, `<p>` headings with no bold text, and headings containing none of `QUALIFICATION_KEYWORDS`.
- ☐ `retrieve_qualification` returns `{}`, not `None` for a page with no qualifications, and works on Greenhouse postings as well as Google's and OpenAI's.
- ☐ `pytest test_hw4_api.py` passes (the offline parsing tests must pass; the live Gemini tests may skip).
- ☐ `retrieve_anthropic_jobs` returns real job postings, not an empty list or a `TimeoutError`.

- ☐ `return_gemini_summaries()` is called once, after the loop, with every qualification — not once per posting.
- ☐ Anthropic results and Vertex AI Search results both appear in the same saved GCS blob (not just one or the other).
- ☐ Removing "Anthropic" from the request's `company_dict` still works and returns only the search-based results.
- ☐ A Vertex AI Search failure still surfaces as a `500` response.
- ☐ `docker build` succeeds for the updated `fastapi/Dockerfile` and `streamlit/Dockerfile`, and the container actually launches a browser successfully at runtime (not just at build time).
- ☐ `bash gcloud_command.sh` deploys both services, and a POST to the **deployed** API writes a new blob to your bucket — not just a local `docker run`.
- ☐ You tore down your Cloud Run services and scheduler job after testing. Leaving them up burns free-tier quota and the job keeps POSTing every night.
- ☐ Submit a zip called **HW4** containing **only these 6 files**, keeping the folder structure:
`fastapi/qualification_parser.py`, `fastapi/anthropic_search.py`,
`fastapi/extract_save_data.py`, `fastapi/Dockerfile`, `streamlit/Dockerfile`, and
`gcloud_command.sh`. `streamlit/Dockerfile` is unchanged from HW3, but
`gcloud_command.sh` deploys from `./streamlit`, so it has to be in the zip for the deployment to run.
- ☐ Skipped tests due to the non-paid gemini API account are fine.